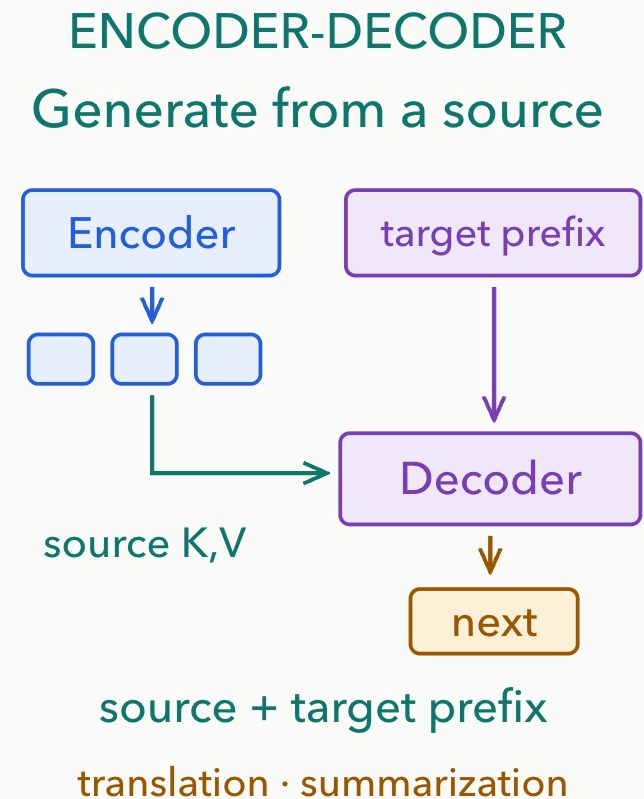
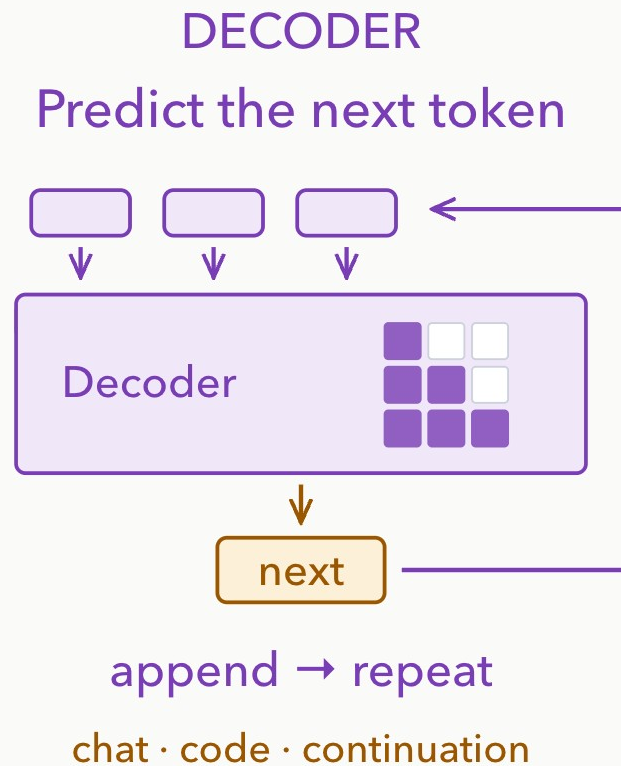
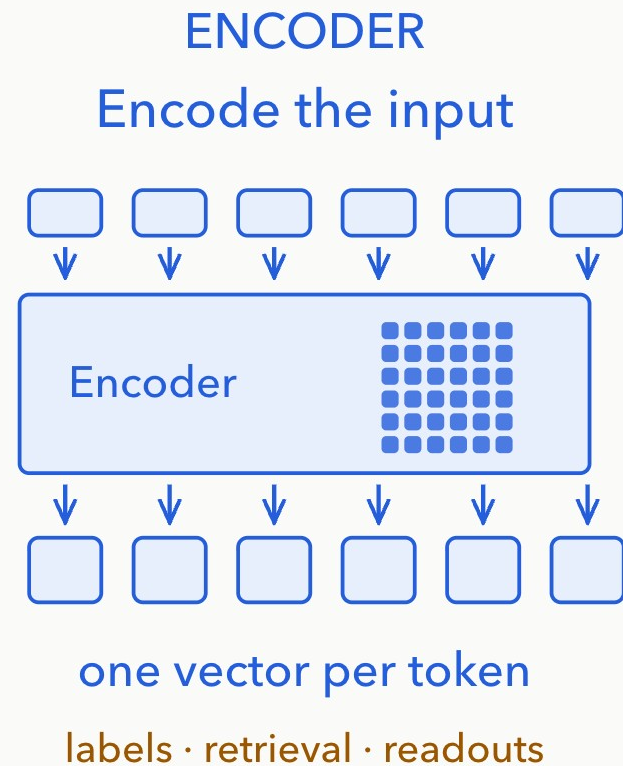


VISION TO LANGUAGE · VISION I

Vision Transformer

Five questions turn one photograph into tokens, context and an ImageNet prediction.

We already know how an encoder builds context



Today we reuse the encoder: the new input is an image.

What should this photograph produce?



Our input photograph



One measured forward pass

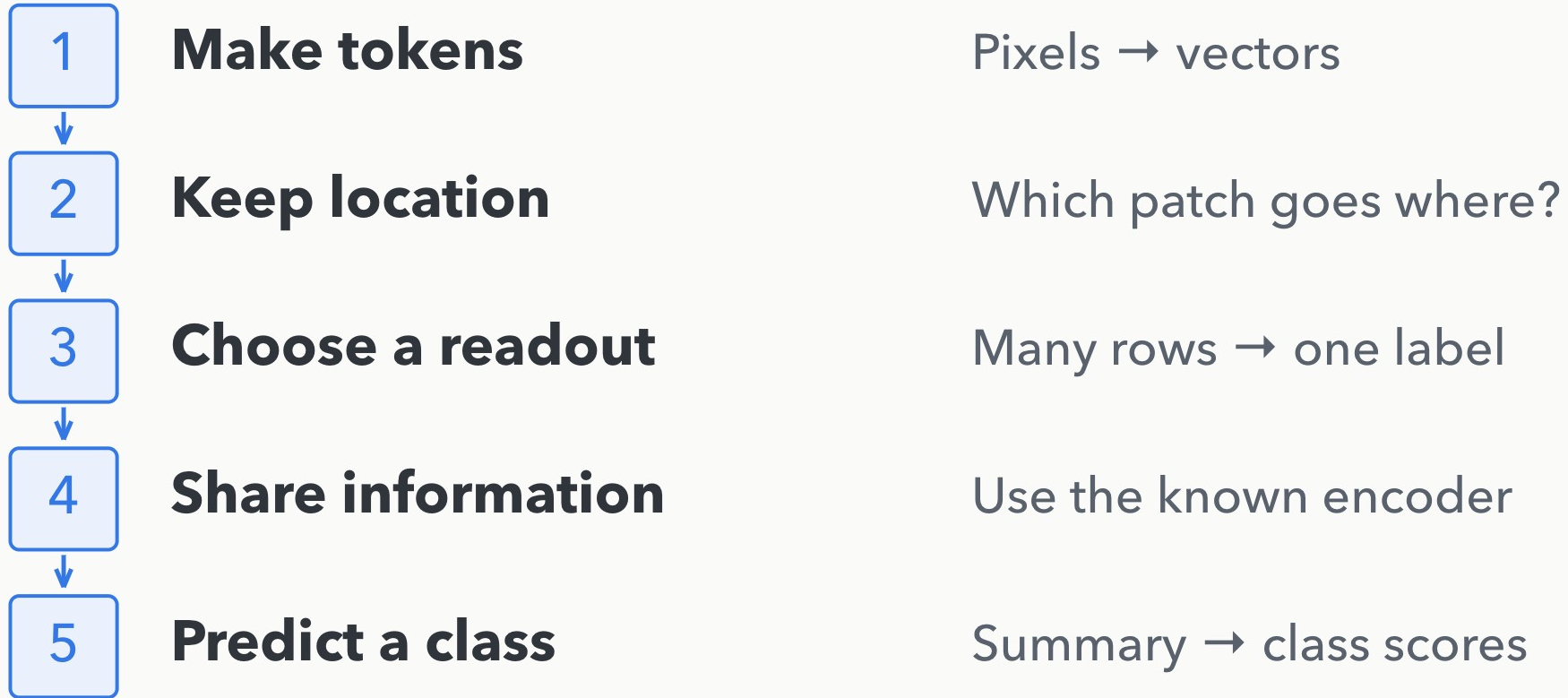
The model predicts one of 1,000 ImageNet classes; Oxford-IIIT Pet supplies the photograph.

An encoder expects vectors. How can an image supply them?



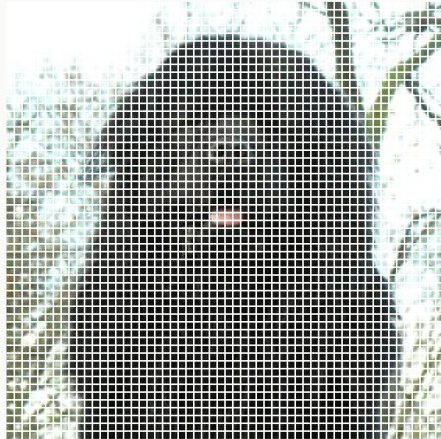
We need a sequence of vectors before we can reuse the encoder.

Five questions build the architecture



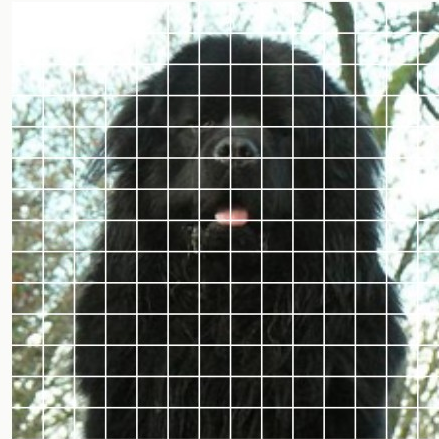
Each new component will solve one problem in this chain.

Why not make every pixel a token?



Pixel tokens

224×224
50,176 tokens



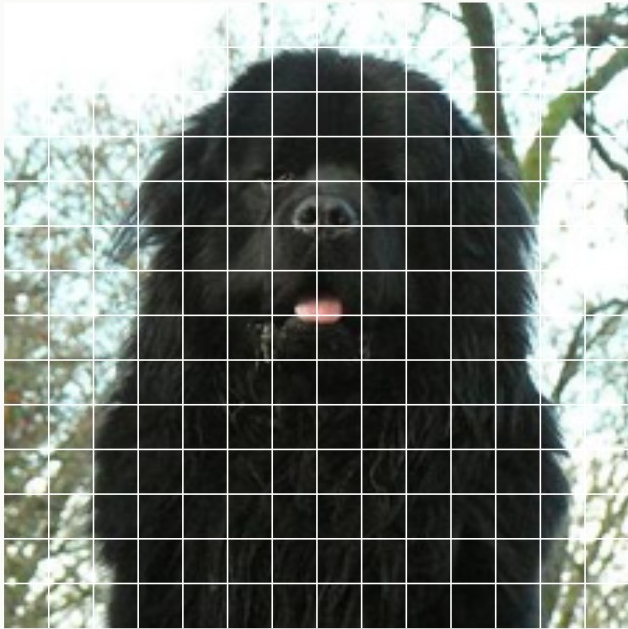
16×16 patches

14×14
196 tokens

Attention compares token pairs: its score matrix grows as N^2 .

Patching is also a computational choice.

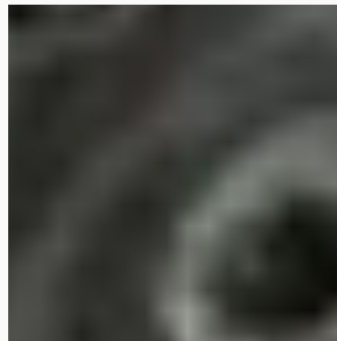
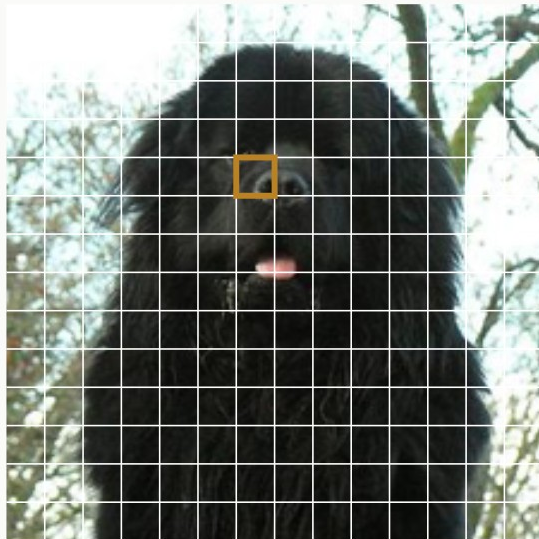
Split the same photograph into 196 patches



$$\begin{aligned} 224 \div 16 &= 14 \text{ patches per side} \\ 14 \times 14 &= 196 \text{ patches} \end{aligned}$$

Each non-overlapping crop is 16×16 pixels, with all three colour channels.

What is inside one patch?



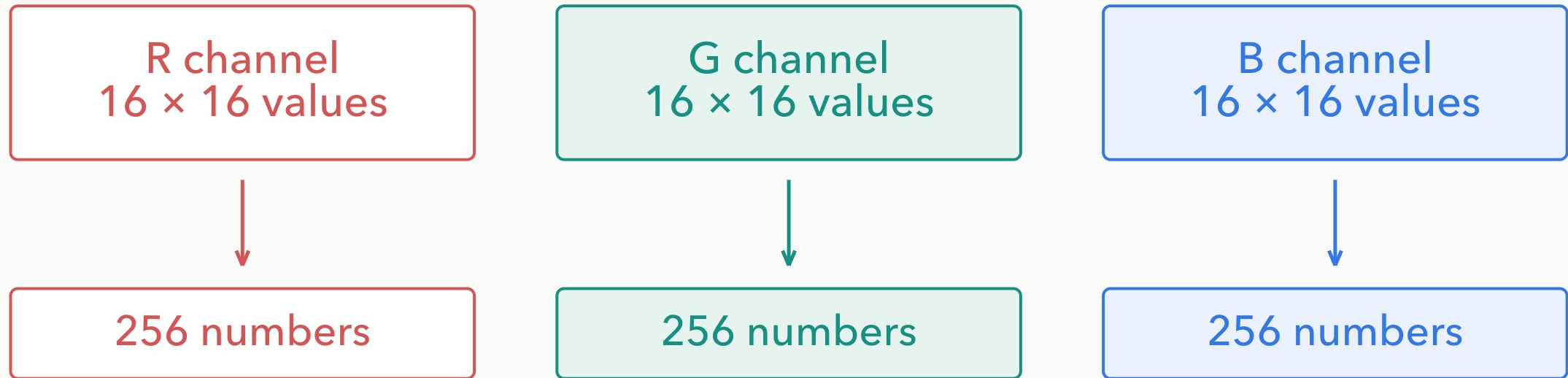
P63



$16 \times 16 \times 3$
768 input numbers

One patch contains 768 pixel values; its position in the image is a separate fact.

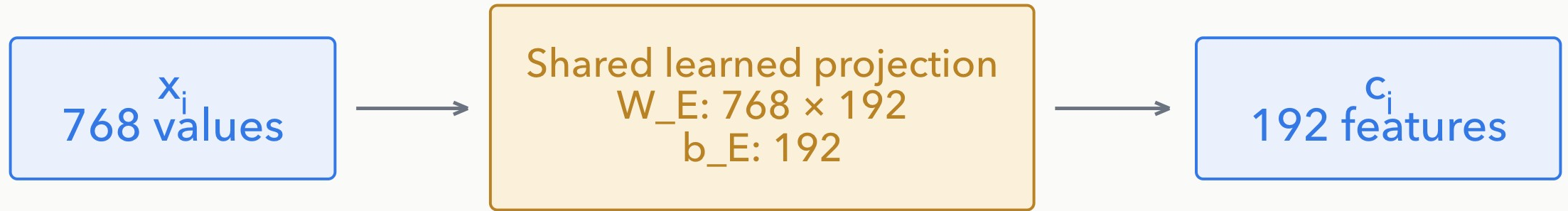
Flatten one channel at a time: R, then G, then B



$$\mathbf{x}_i = [\text{all R pixels} \mid \text{all G pixels} \mid \text{all B pixels}] \in \mathbb{R}^{768}$$

Flattening only rearranges the values; it does not learn features.

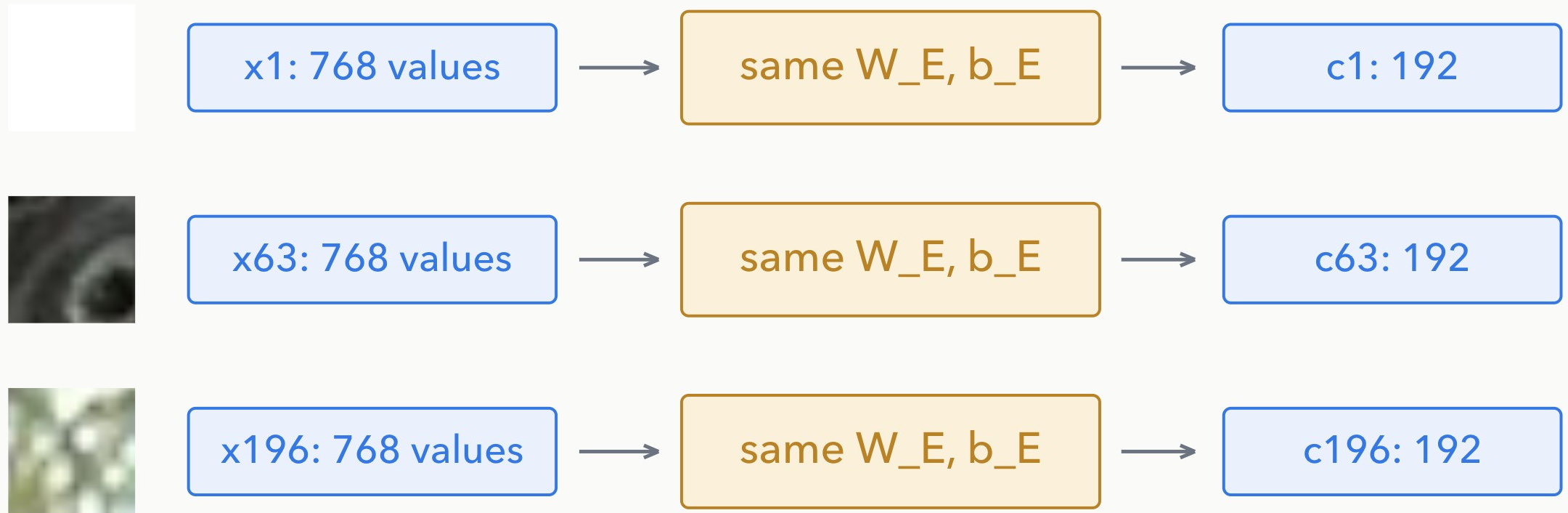
How do 768 pixel values become 192 features?



$$c_i = x_i W_E + b_E$$

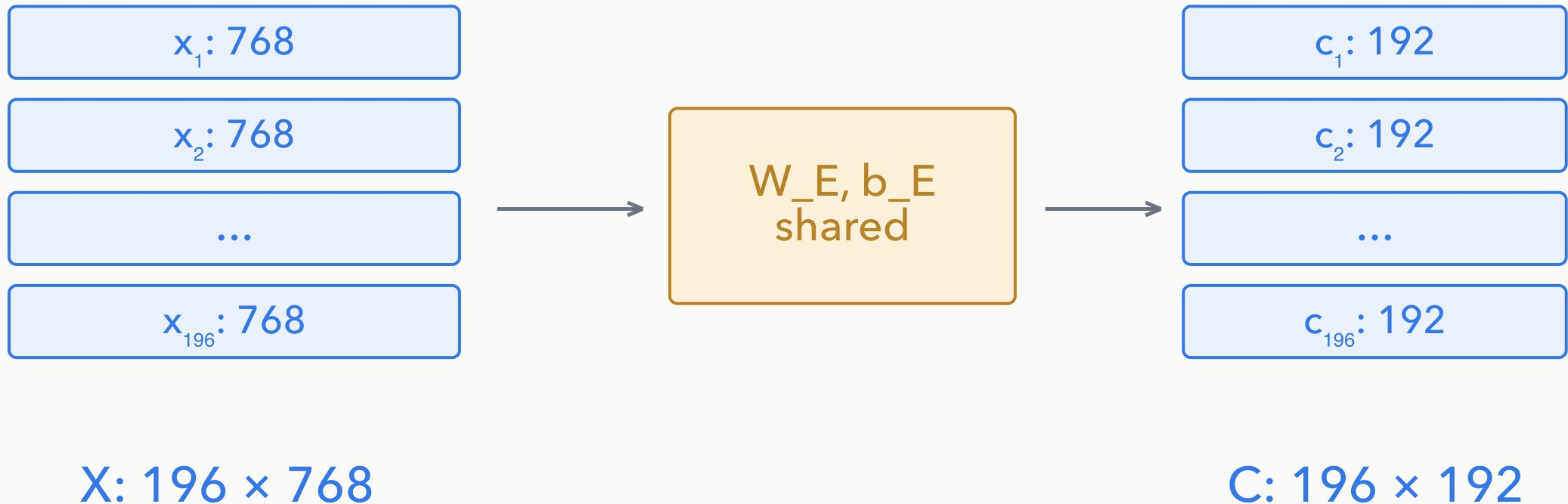
A learned linear projection computes the patch representation.

Should every patch get a different projection?



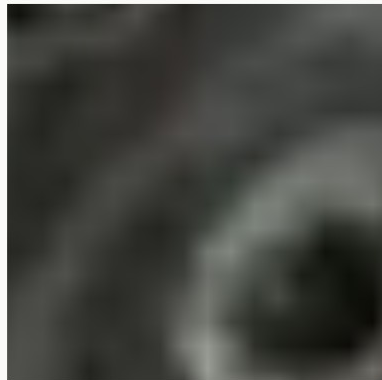
The same W_E and b_E are reused at all 196 locations.

Stack the patches into one feature matrix



Projection changes the feature width; it preserves the number of patch rows.

What does the trained projection actually return?



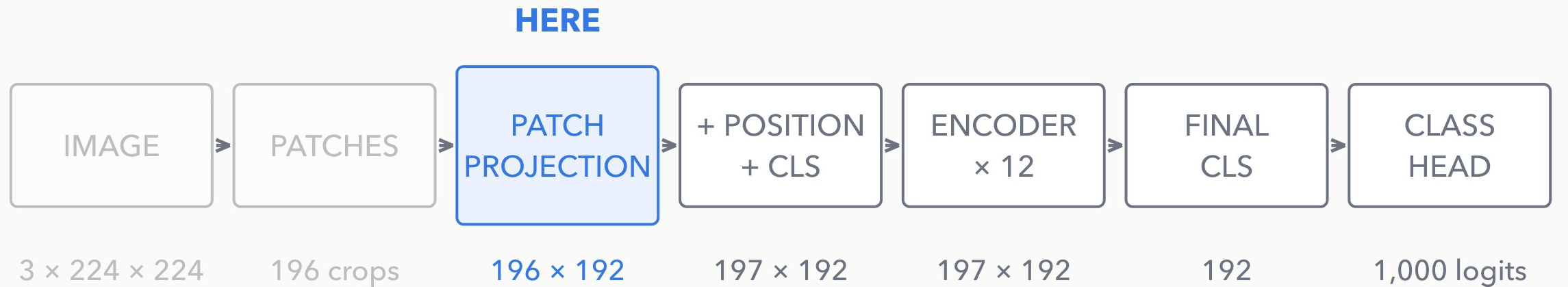
Measured P63



$[-0.852, 1.339, 0.504, \dots]$

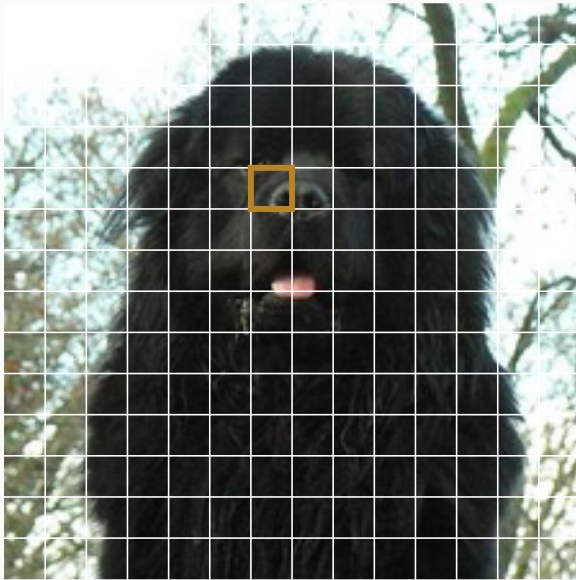
P63 becomes a measured 192-number feature row.

We have solved the image-to-token problem



The image now supplies 196 rows of 192 learned features.

The projection is shared. Where does location enter?



content c_{63}

Where?
row 5, column 7

Content describes the crop; a positional vector supplies its place in the image.

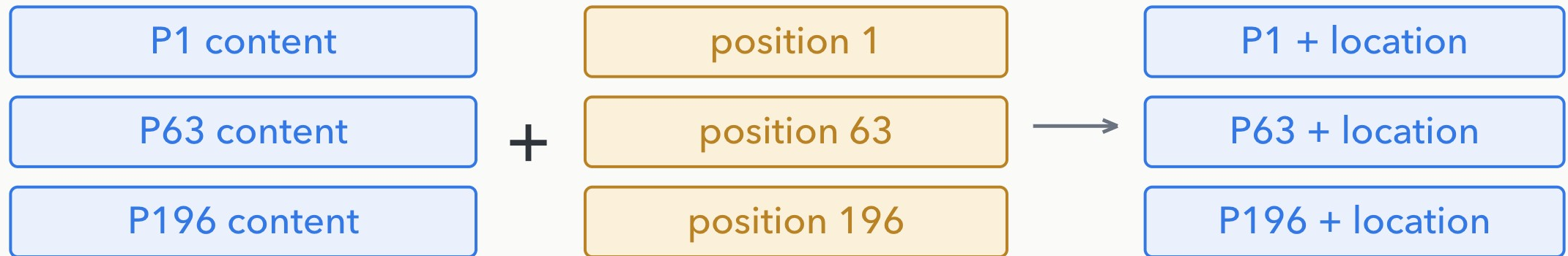
Add WHAT and WHERE



$$e_i = c_i + p_i \cdot 192 \text{ features}$$

Adding position preserves the 196×192 shape.

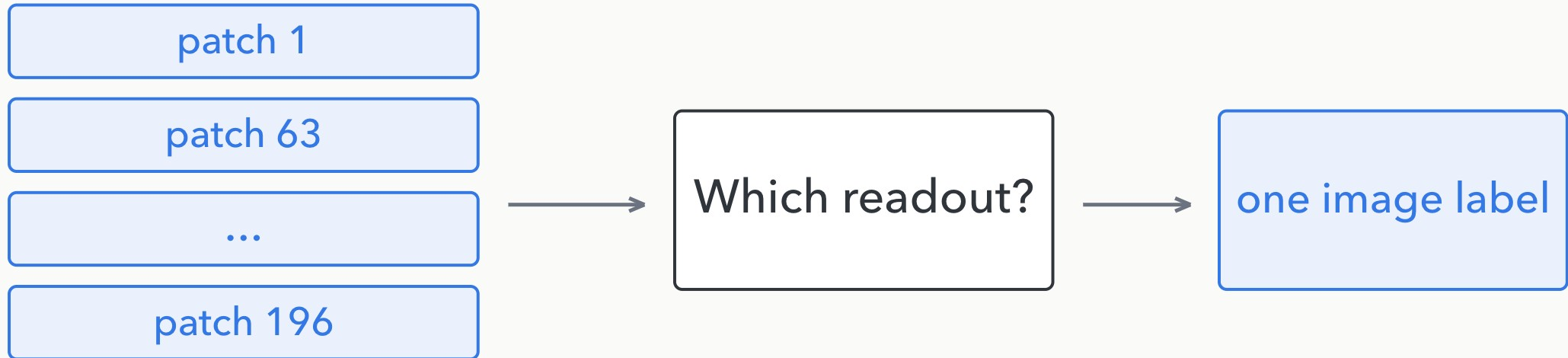
Which positional vector goes with each row?



Learned table: one 192-number row per token position

The same learned position table is reused for every image on this grid.

Many patch representations, one image label



Mean pooling is another valid readout; this checkpoint was trained with CLS.

Reuse the CLS readout we already know from text.

Same CLS idea, different input tokens



TEXT

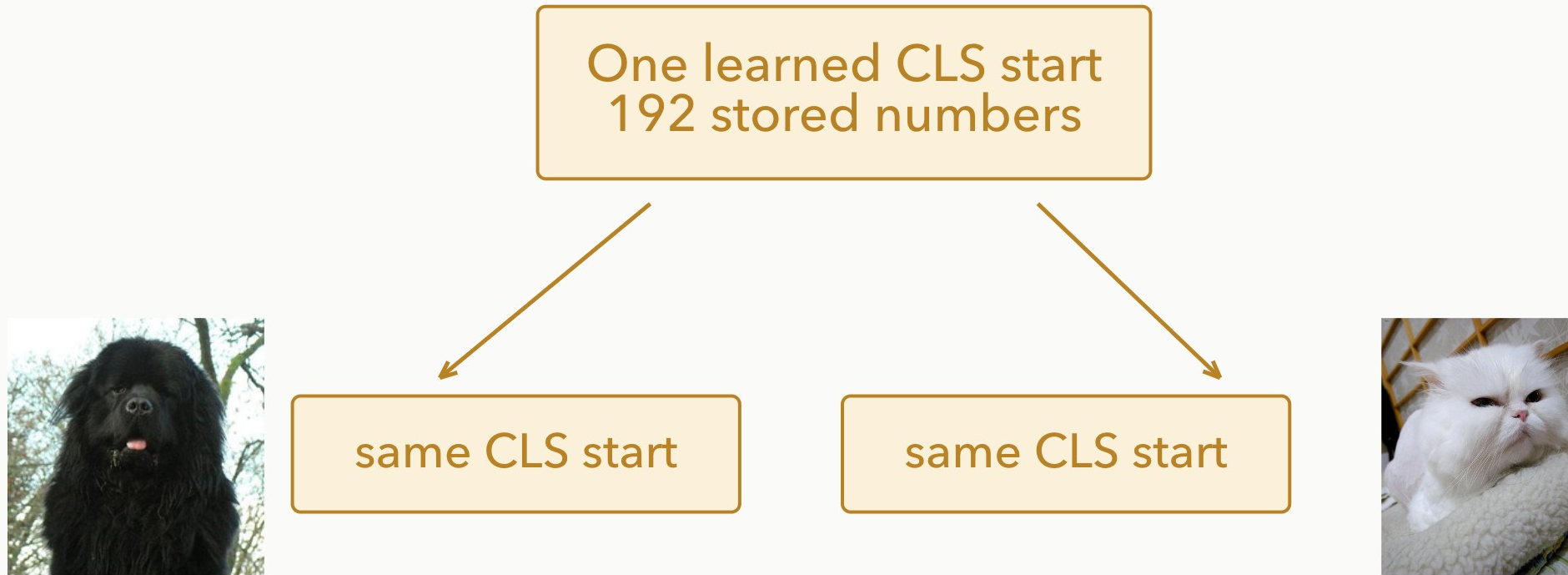


IMAGE



The image supplies patch tokens where the text model supplied word tokens.

Every image starts with the same learned CLS vector



The stored CLS parameter is shared across photographs.

After reading the image, CLS becomes image-dependent



same CLS start

same encoder
+ these patch rows

dog image summary



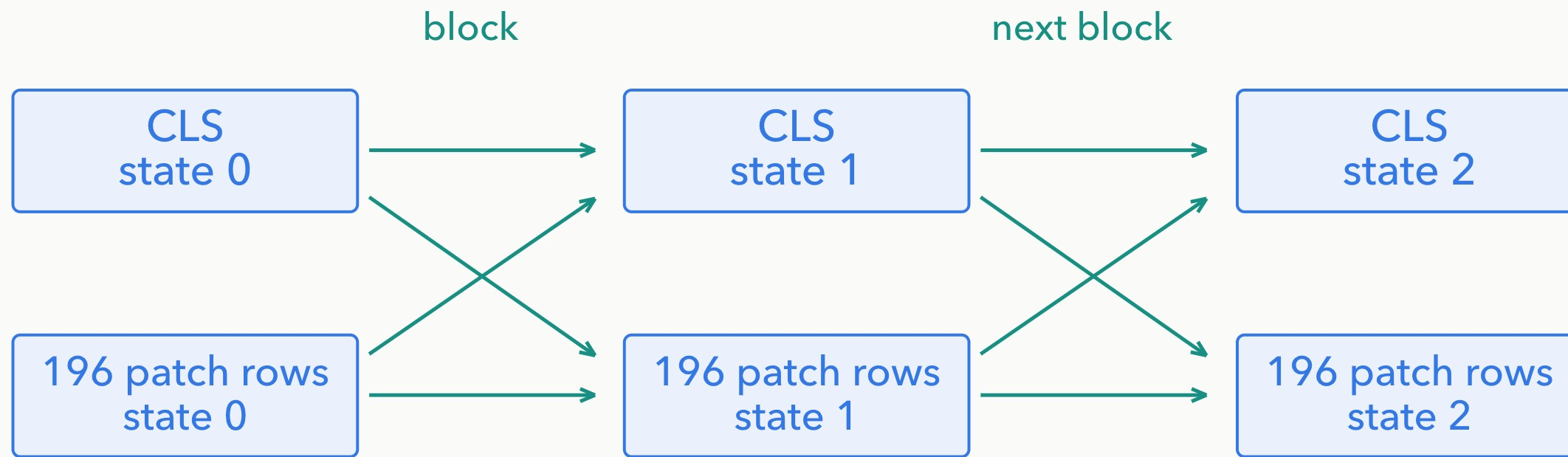
same CLS start

same encoder
+ these patch rows

cat image summary

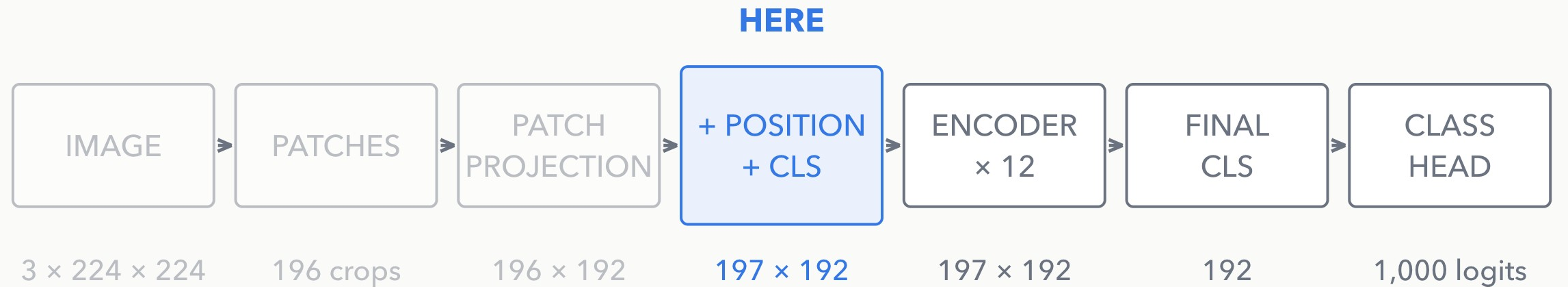
Shared parameters can produce different representations for different inputs.

CLS reads the current patch states at every block



CLS and patch rows can exchange information in both directions.

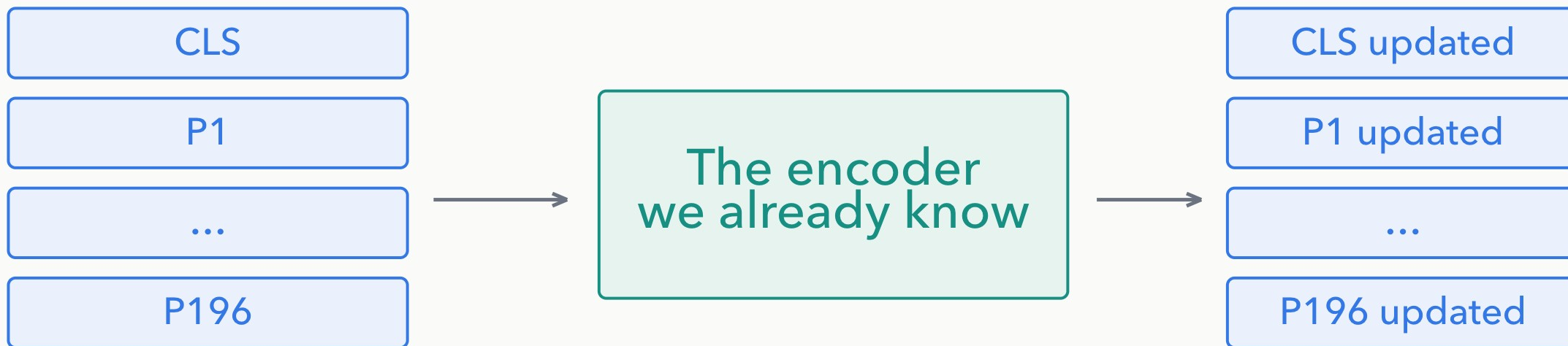
We now have the token sequence the encoder needs



[CLS ; P1 ; P2 ; ... ; P196] + position → 197 × 192

196 projected patches plus one CLS row, each with 192 features.

From here, reuse the encoder we already know



The new work was turning an image into tokens.

Where do Q, K and V come from?



TRANSLATION • cross-attention

target-prefix states $\rightarrow$ Q

source-encoder states $\rightarrow$ K, V

ViT • self-attention

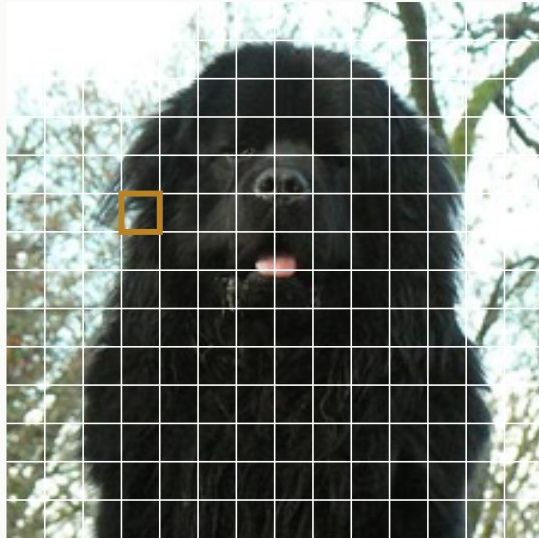
CLS + patch states
one image sequence



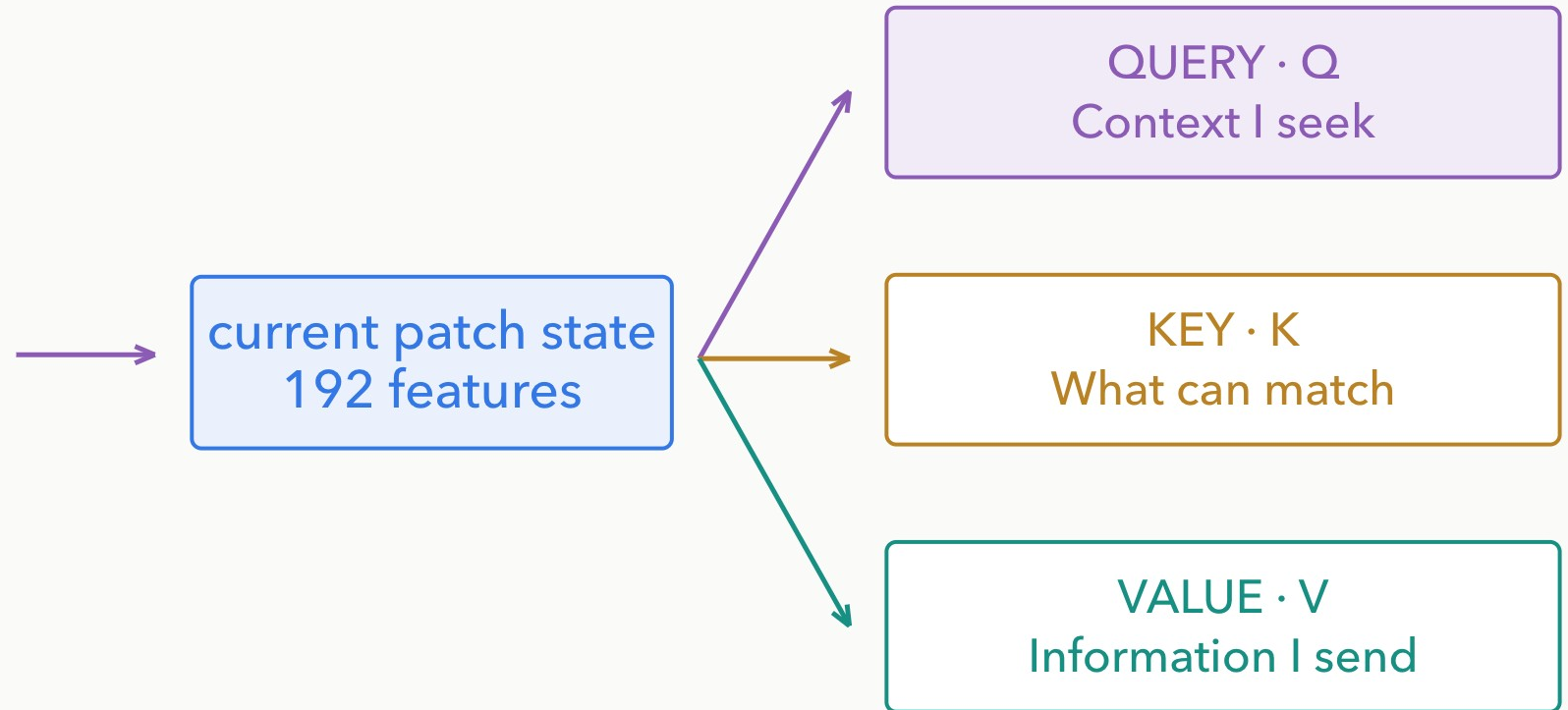
Q, K and V

ViT has one stream; translation cross-attention had two.

One patch representation has three jobs



Receiver: P74

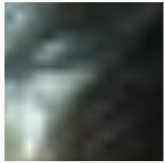


Every token makes a query, a key and a value through three learned projections.

What might different queries try to gather?



Illustrative questions – the model uses vectors, not sentences



A patch on the dog

Which other regions help interpret this texture?

Other fur / face regions



A background patch

Which other regions provide scene context?

Branches / bright background



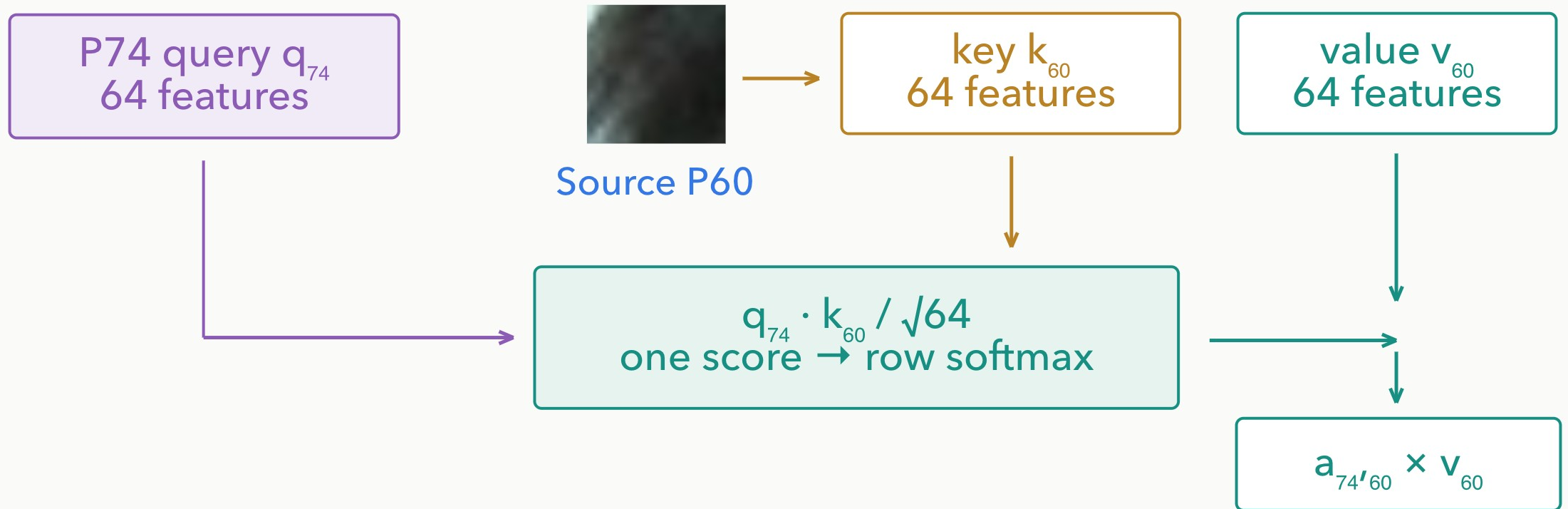
CLS

Which image features help classify the whole image?

Useful parts across the photograph

A different receiver can ask for different context from the same image.

A source supplies both a key and a value



Q and K set a scalar weight. V supplies the vector being weighted.

The weight for P60 scales all 64 coordinates of P60's value vector.

Several source values form one receiver's message



Toy example · one receiver · three sources · two value features



Add the contributions: $m = [1.3, 0.4]$

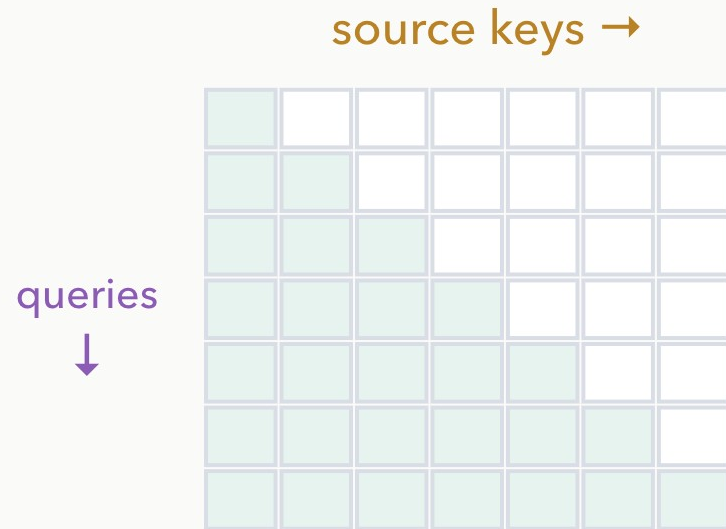
This message goes to the receiver whose query chose the weights.

Next, next, next: weight each source vector, then add the contributions.

ViT uses full attention, not causal attention



TEXT DECODER • causal



Earlier + current tokens

ViT ENCODER • full

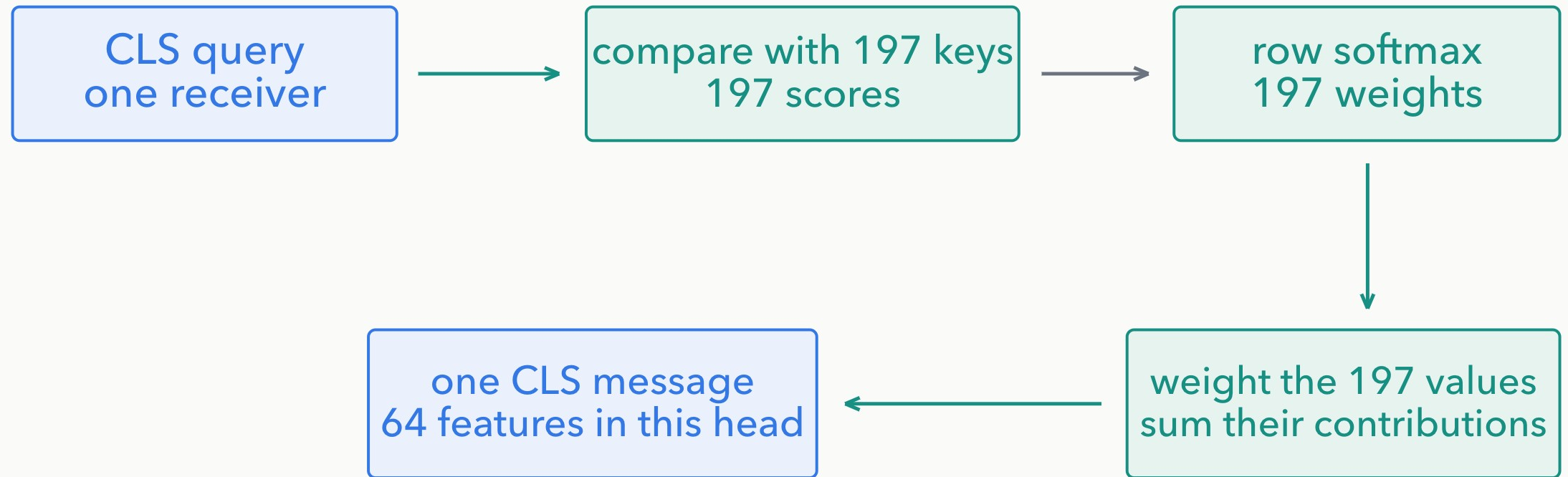


CLS + every patch token

Filled = allowed. ViT sees the complete image: no causal mask.

Rows are receiving queries; columns are source keys. Allowed does not mean equally weighted.

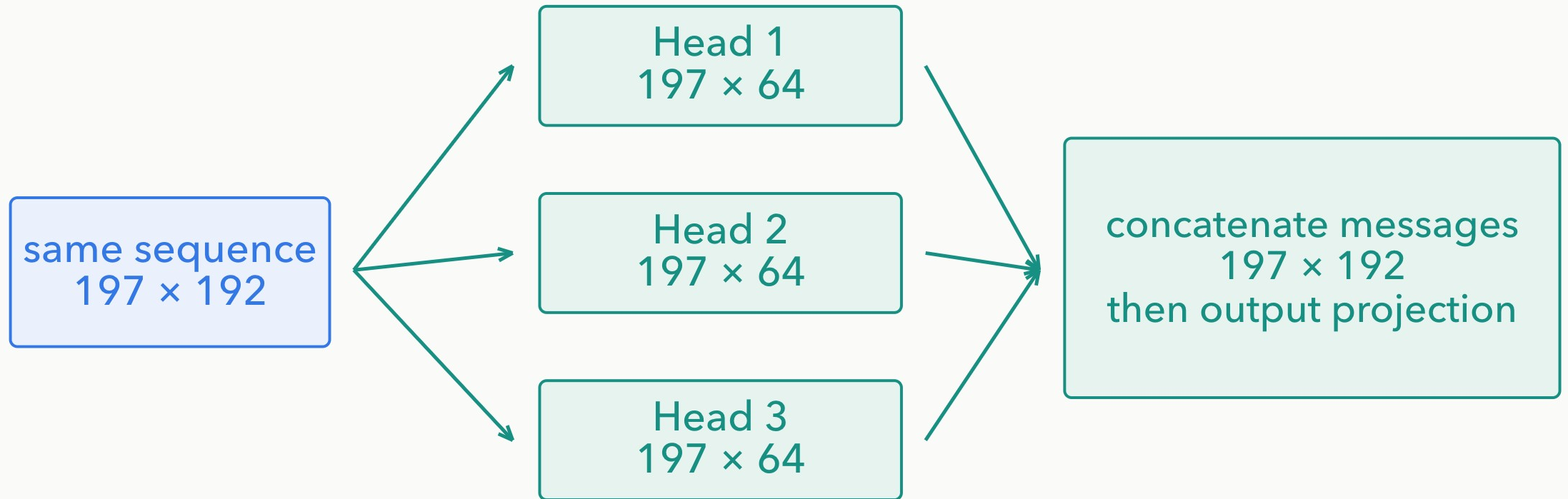
How does one CLS query collect one message?



Other query rows receive their own messages in parallel.

The message belongs to the query's row: here, CLS.

Three heads form three views of the same sequence

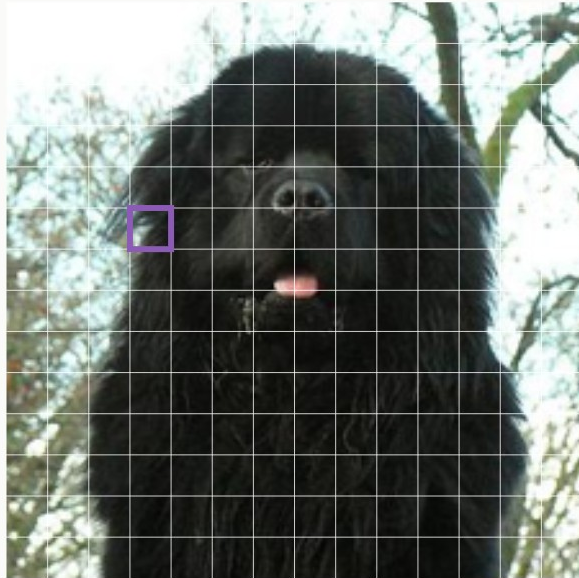


Join each row's three 64-feature messages, then project them back to 192 features.

Where does the trained query look?

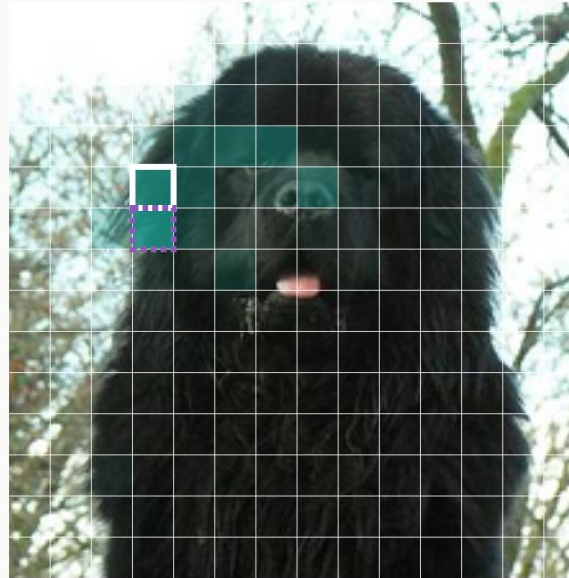


Query: P74

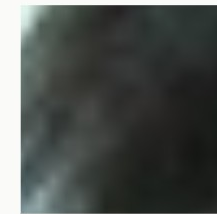


Trained ViT · block 4 · head 1

Measured source weights



Teal: 0 → 10.35%



P60

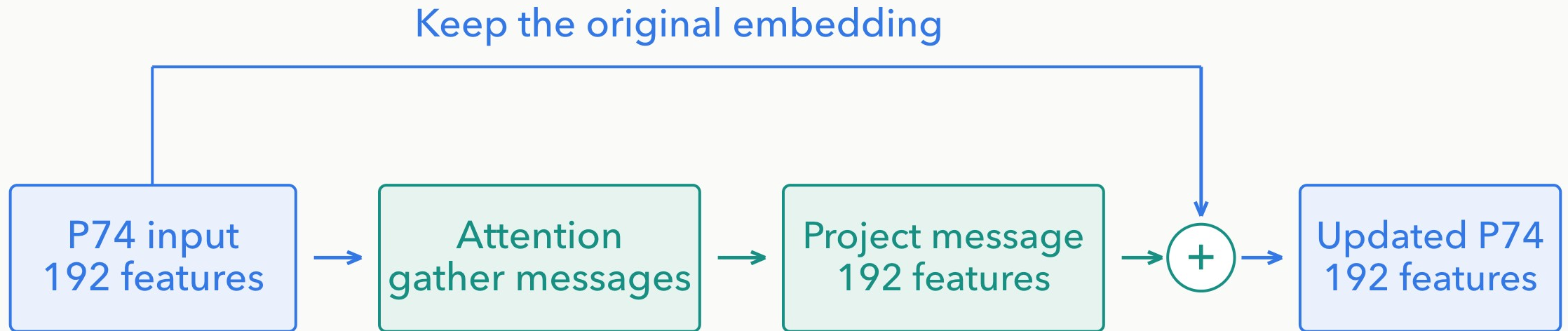
Weight: 9.09%

P60's value contributes to P74's message.

[Explore the nine examples ↗](#)

Fix one query and inspect its source weights; other heads and blocks can gather different mixtures.

Add the attention update to the original embedding

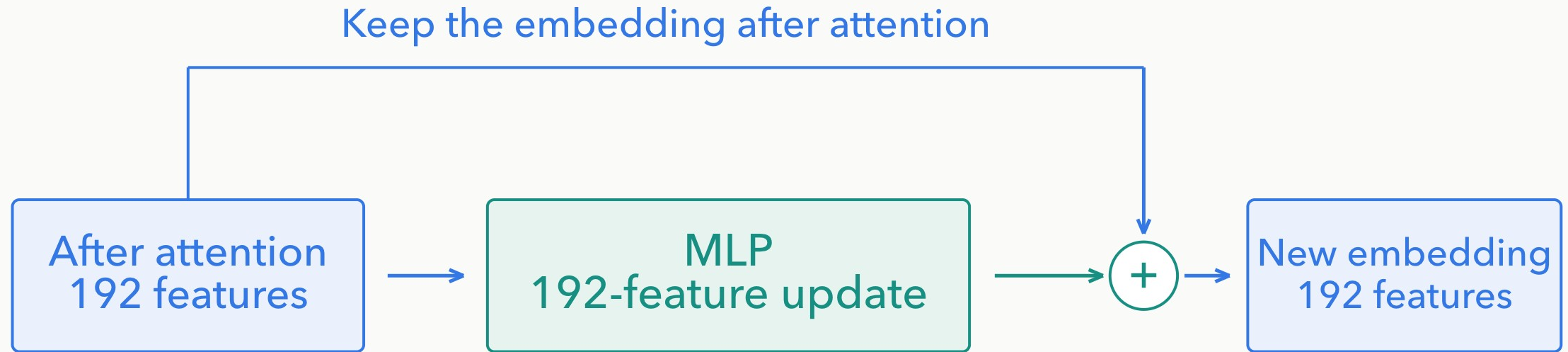


original embedding + projected message = updated embedding

Every patch gets its own update. So does CLS.

Gather a message, project it to 192 features, then add it to the receiving token's original embedding.

The MLP adds one more update to each embedding



The MLP processes each token separately.

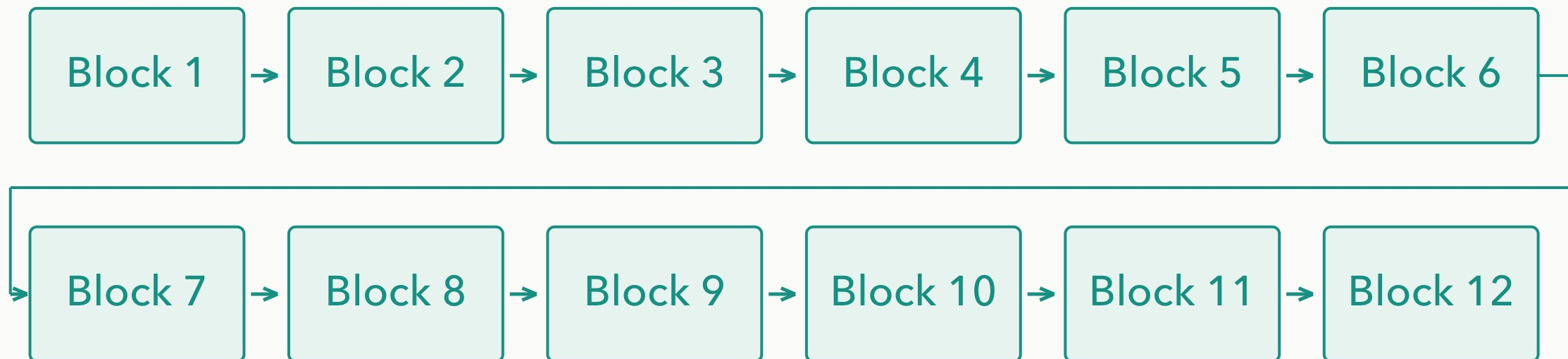
This new embedding is ready for the next block.

Attention gathers context from other rows; the MLP then processes each row's features.

Every block updates the patches and CLS again



Each block: attention update, then MLP update.



197 token embeddings · still 192 features each

The next block starts from these new embeddings. After block 12, we read CLS.

What is stored, and what changes with the image?



LEARNED PARAMETERS

Patch projection W, b

Position table + starting CLS

Transformer weights, including $W_Q/K/V$

Classifier weights + biases

COMPUTED FOR EACH IMAGE

Patch embeddings

Q, K, V and attention weights

Contextual patch states + final CLS

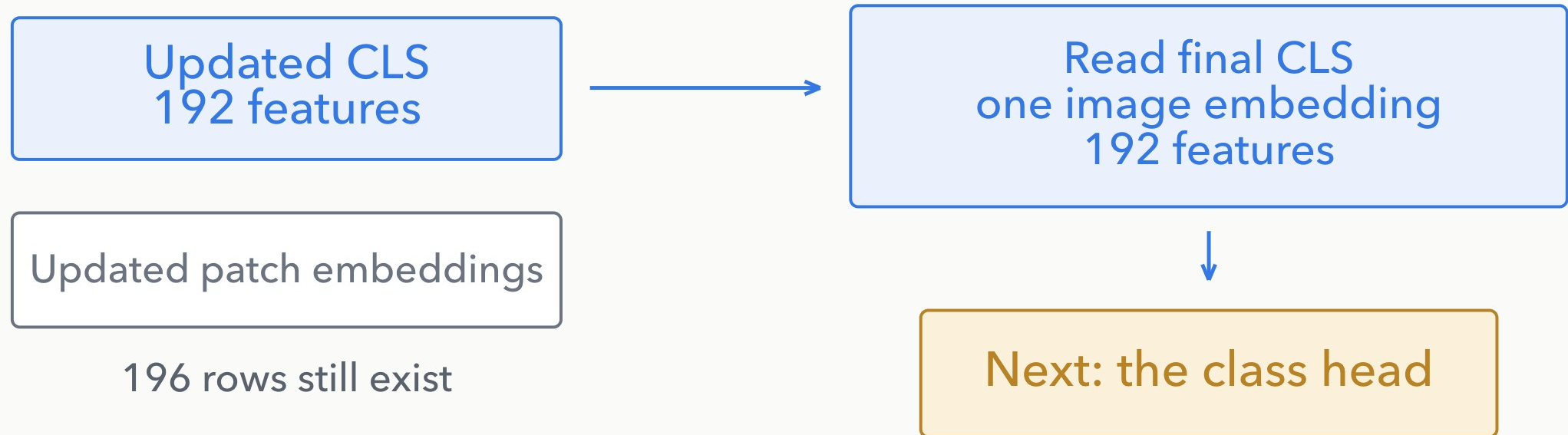
Logits + probabilities

W_Q, W_K and W_V are learned; attention weights are computed separately for each input.

After the blocks, read the updated CLS embedding



After all 12 blocks



The classifier uses the image summary carried by CLS.

One final CLS embedding summarizes the image for the classifier.

How do 192 features score 1,000 classes?



One logit and one probability for every ImageNet class

The class head learns how each image feature contributes to each class score.

What does this checkpoint predict for our photograph?



Newfoundland **95.73%**



Tibetan mastiff **1.63%**



briard **0.67%**

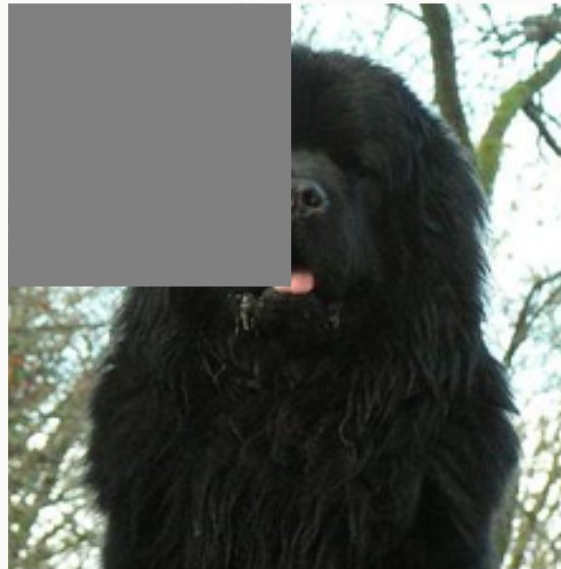
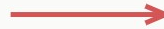


Newfoundland receives 95.73% of the probability across the 1,000 classes.

What happens if we hide one quarter of the image?



Original: 95.73%



Cover one quarter

Same model.
Changed pixels.

Same label?
Same probability?

Predict the effect before we run the changed image through the same trained model.

Replace pixels, then recompute the whole forward pass



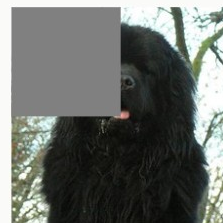
```
covered = x.clone() # x: (1, 3, 224, 224), normalized
covered[:, :, :112, :112] = 0 # gray RGB; 112 × 112 pixels
model.eval()
with torch.inference_mode():
    before = model(x).softmax(-1)[0, 256]
    after = model(covered).softmax(-1)[0, 256]
```



same ViT



95.73%



ViT



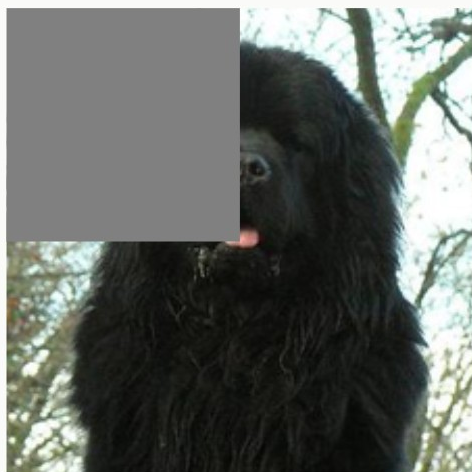
83.00%

Weights stay fixed; all patch features, attention weights and class probabilities are recomputed.

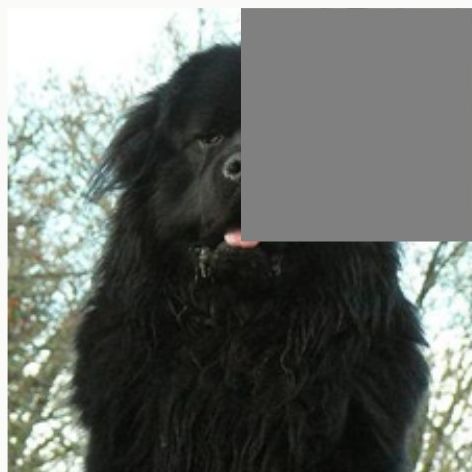
Four independent covers, four measured predictions



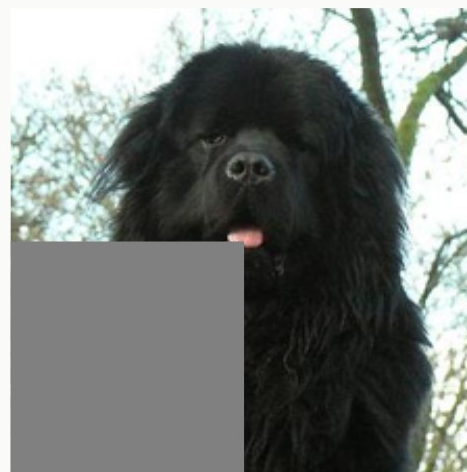
Original $P(\text{Newfoundland}) = 95.73\%$



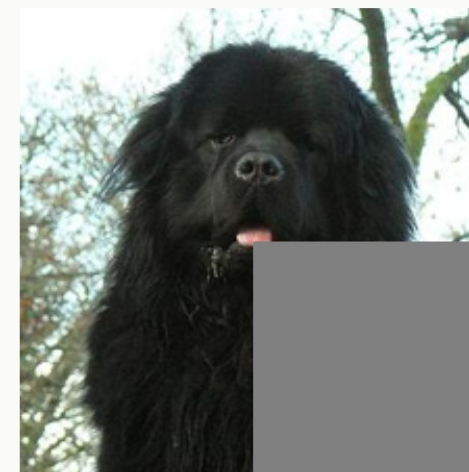
Top left
83.00%
-12.73 points



Top right
79.62%
-16.11 points



Bottom left
84.54%
-11.19 points



Bottom right
82.16%
-13.57 points

Top right causes the largest drop: 16.11 points. All four images still predict Newfoundland.

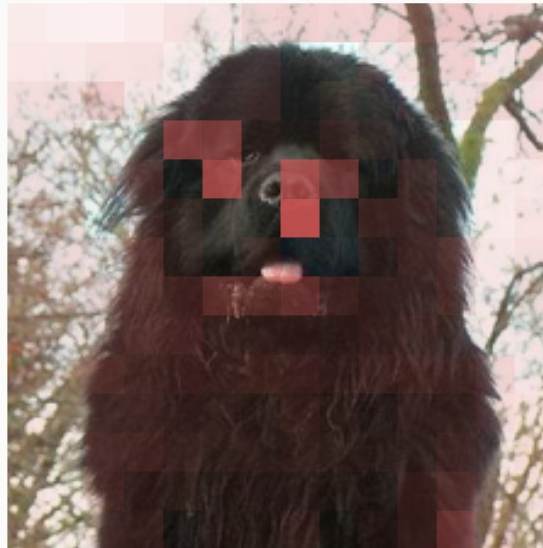
Would smaller covers tell us more?



Cover one patch: P78



196 separate tests



Largest local drop

95.73% → 91.89%

P78: -3.84 points

Red: probability falls

Blue: probability rises

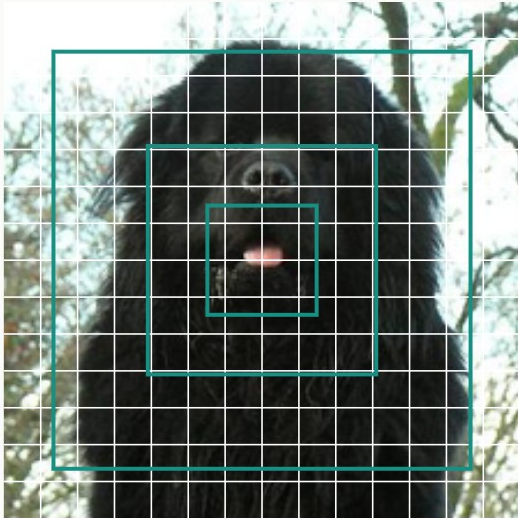
Scale: -4 to +4 points

All 196 tests keep the same label. Smaller covers probe more local sensitivity.

Each coloured square records a separate forward pass with one 16×16 patch covered.

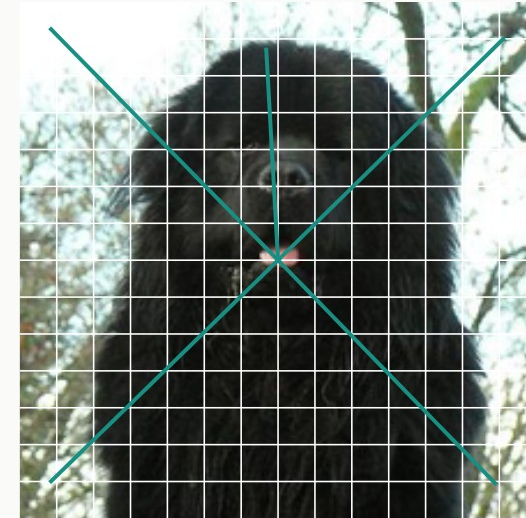
Two ways to gather image context

CNN



Nearby first → wider context

ViT



Direct access to distant patches

CNN context grows through local layers; global ViT attention can connect distant patches within one block.

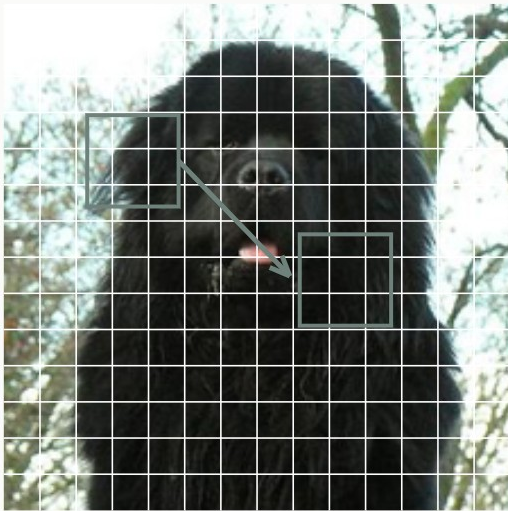
Which assumptions are built into the architecture?

	CNN	ViT
Basic units	Local feature maps	Patch-token rows
Mixing rule	Shared local convolution	Content-dependent attention
Locality	Built into each filter	Less spatial structure built in
Position	Grid structure	Explicit position embeddings

Inductive bias is the structure supplied before learning begins.

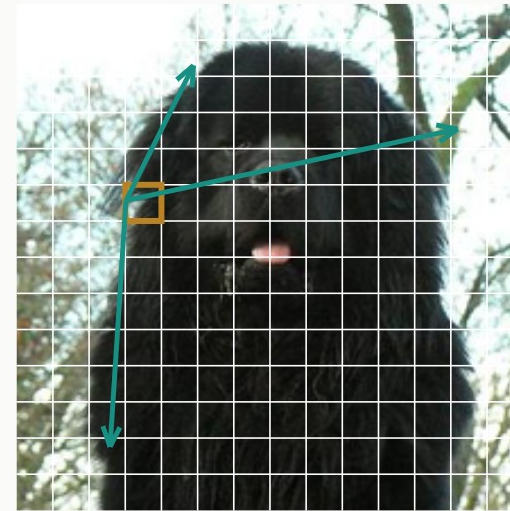
What does an inductive bias buy us?

CNN: reuse a local detector



Same filter, different locations

ViT: choose context from content



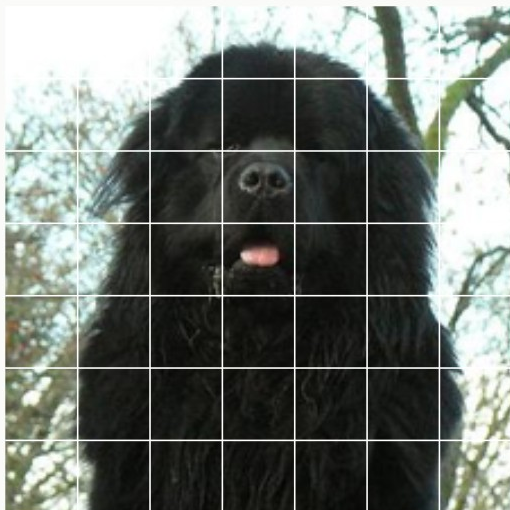
Different weights for each query

A built-in image prior can help with limited data; pretraining changes the comparison. Locality and filter sharing make useful image assumptions before the CNN sees any examples.

How much detail should one token cover?

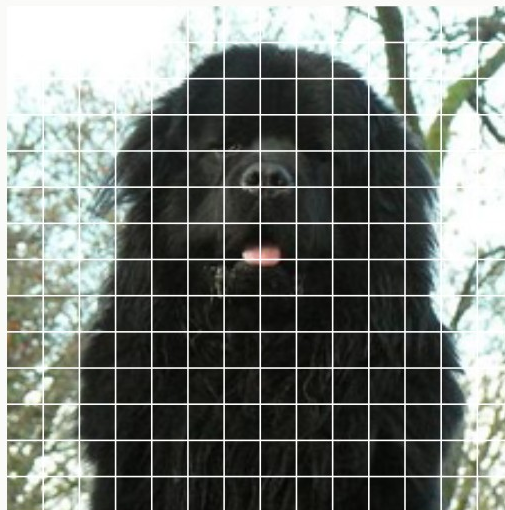


32 × 32 patches



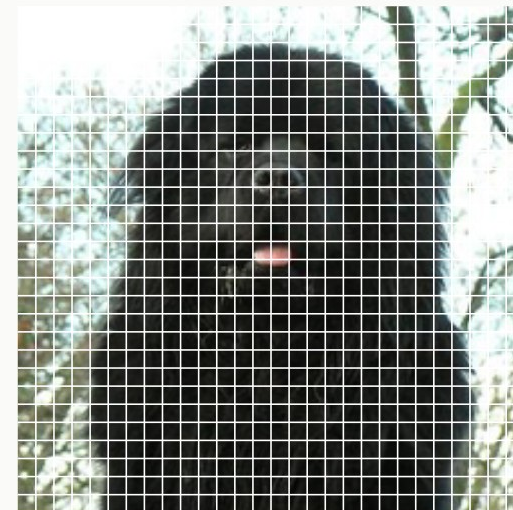
49 patch tokens

16 × 16 patches



196 patch tokens

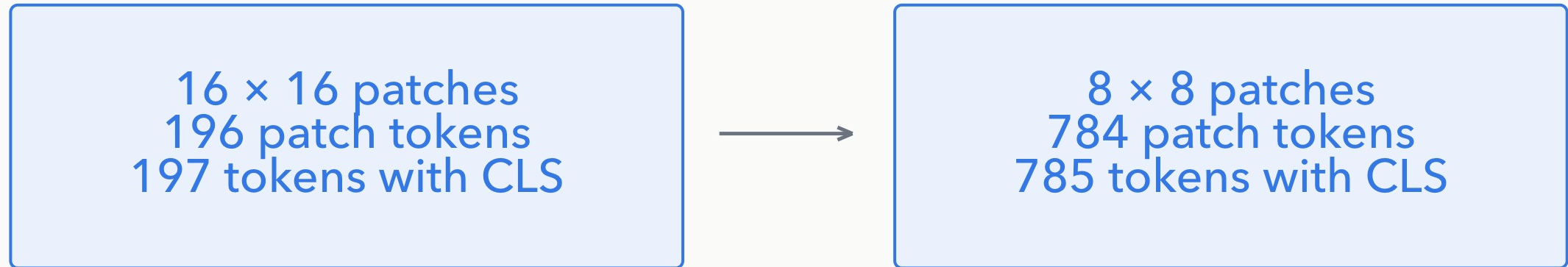
8 × 8 patches



784 patch tokens

Smaller patches preserve a finer spatial grid and create more tokens.

What does a finer grid cost?

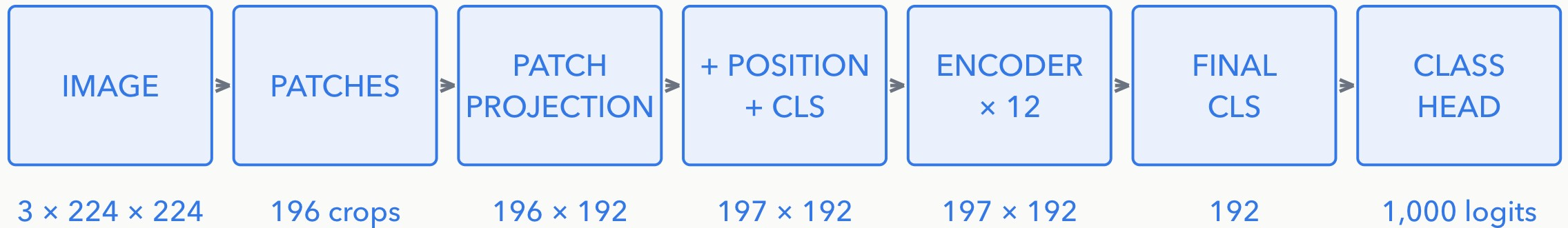


Patch tokens × 4 → attention scores ≈ × 16

38,809 scores → 616,225 scores per head

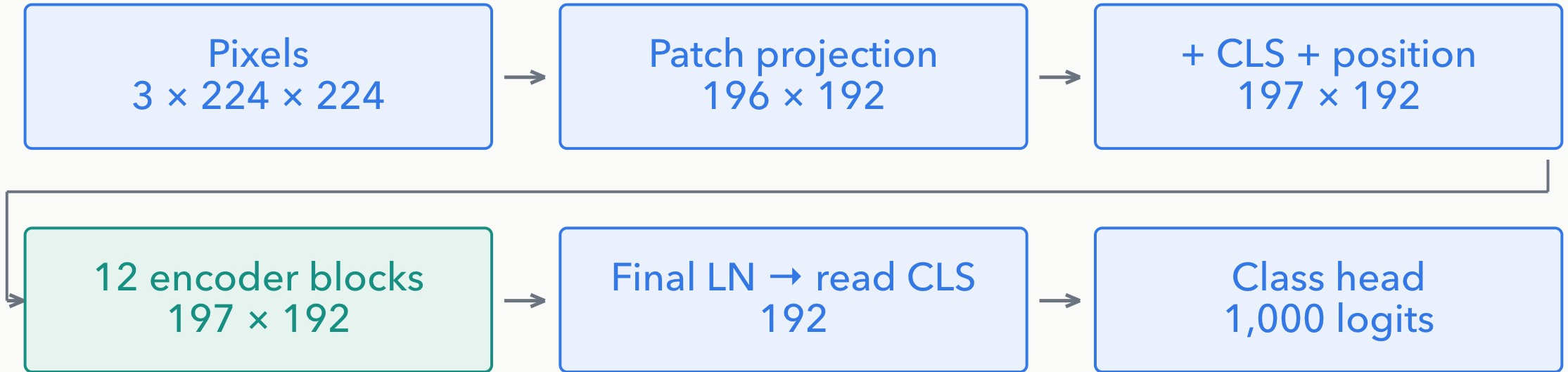
The pairwise score count grows quadratically with token count.

Once an image becomes tokens, the encoder is familiar



Patches make tokens; the known encoder builds context; the class head reads the image summary.

Follow the whole model through its shapes



**BLOCKS PRESERVE THE SHAPE;
THEY CHANGE WHAT EACH ROW REPRESENTS.**

Batch dimension omitted; all shapes refer to one photograph.

The whole ViT in six lines

```
x = patch_embed(image)           # B, 196, 192
x = prepend_cls(x) + pos          # B, 197, 192
for block in blocks:
    x = block(x)                  # B, 197, 192
h = norm(x)[ :, 0 ]              # B, 192
logits = head(h)                 # B, 1000
```

The optional implementation lab opens each of these operations.

Choose a deeper dive when you need it

OPTIONAL LABS & EXTENSIONS

Implementation lab

Extensions

Open the boxes

Linear $\leftrightarrow$ Conv2d

Transfer + evaluation

Patch arithmetic

PyTorch shapes + code

Similarity · attention · occlusion

Real attention numbers

The conceptual model is complete; these are optional deeper dives.

The reference deck preserves the detailed diagrams, code, measurements and exercises.

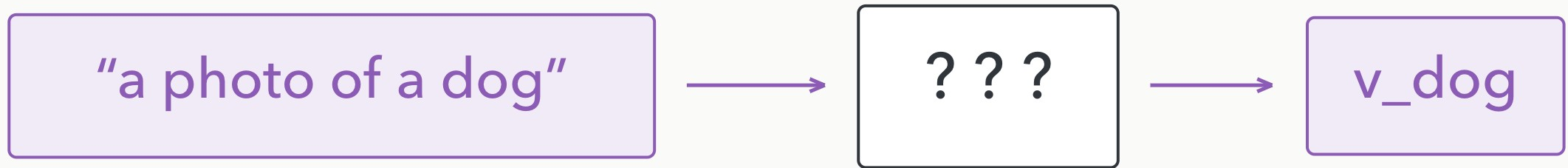
The classifier stores a learned vector per known class



$$s_c = h_{CLS}^T w_c + b_c$$

Its 1,000 output classes form a fixed vocabulary.

What if our class vocabulary could come from words?



What if a class vector came from language?

NEXT: CLIP

The next lecture begins with this question.